

FACULTY OF MEDIA ENGINEERING AND TECHNOLOGY

German University in Cairo

CSEN 601 – Computer Systems Architecture

Spring 2026

Project Report

Package 4: Double McHarvard with Cheese Circular Shifts

Team Number	Team_06
Package Number	4
Package Name	Double McHarvard with Cheese Circular Shifts
Responsible TA	Nourhan Elgendy
Submission Date	Monday, 18 May 2026

Team Members

Name	Student ID	Tutorial
Haya Bakier	61-15631	30
Yassmin Kamha	61-6566	30
Hana Issa	61-15832	30
Toqa Abdalla	61-15539	30
Shahd Walid	61-13378	30
Layal Maged	61-3030	24

1. SREG-REGISTERS

In code, the **registers** are represented by the registerFile array, which contains 64 storage locations corresponding to R0 through R63. They are used like the processor's temporary working memory: instructions take their input values from registers, perform an operation, and usually place the result back into a register. For example, when the program executes an instruction like `ADD R1 R2`, it retrieves the current values stored in R1 and R2, adds them together, and saves the result in R1. Another example is `LDI R3 10`, where the value 10 is directly loaded into R3. This makes the registers essential for carrying out arithmetic, logical, memory, and branch-related operations throughout the simulation.

The **SREG** is represented in the code by the variable `sreg`, and it works like a small report of what happened after certain instructions are executed. Instead of storing normal data, it stores five status flags: **Carry**, **Overflow**, **Negative**, **Sign**, and **Zero**. For example, if an arithmetic instruction produces the value 0, the code sets the **Zero flag** to 1. If the result is negative, the **Negative flag** becomes 1. If an addition produces a value that is too large to fit in 8 bits, the **Carry flag** is updated. These flags are important because they describe the outcome of operations and help the processor keep track of special conditions. In the code, dedicated helper functions update these flags whenever required, and another function ensures that the unused upper three bits of `sreg` remain cleared, leaving only the five required flags active.

2. Memory&Parser

Package 4 follows the Harvard architecture model, meaning that instruction memory and data memory are separated. In the implementation, the instruction memory is represented by the array `instructionMemory[1024]`, where each location stores one 16-bit encoded instruction. The data memory is represented by `dataMemory[2048]`, where each location stores an 8-bit value. The instruction memory is used only for program instructions, while the data memory stores values accessed through memory instructions such as LB and SB. This separation reflects the Harvard architecture requirement described in the package specifications and allows instructions and data to be handled independently during execution.

The parser is responsible for reading the assembly program from the text file `program.txt` and converting each instruction into its binary encoded representation before execution begins. Instead of storing instruction fields separately, the parser directly encodes every instruction into a single 16-bit machine instruction and stores it in instruction memory. This follows the project requirement that decoding must happen later during the decode stage using masking and shifting operations.

To perform the encoding, two helper functions were implemented: `encodeRType()` and `encodeIType()`. The R-type format stores the opcode in the upper 4 bits, followed by two 6-bit register fields, while the I-type format stores the opcode, one register operand, and a 6-bit immediate value. Bitwise shifting and masking operations are used to place every field in its correct position inside the 16-bit instruction. For immediate instructions, the immediate value is masked using `0b111111` to ensure that only the lower 6 bits are stored. This allows signed immediate values to be represented correctly using two's complement representation.

The parser uses `sscanf()` to match each assembly instruction format directly from the text file. Once the operands are extracted, the instruction is immediately encoded and stored into instruction memory without preserving separated instruction fields. Later, during the decode stage, the encoded instruction is fetched from memory and decoded again using bit masking and shifting to extract the opcode, registers, and immediate values needed for execution.

3. Fetch & Decode

Fetch: Reads the next instruction from program memory using the current PC value, then increments the PC by 1 to point to the next instruction. It loads the 16-bit instruction into the Fetch/Decode pipeline latch so it can be processed in the decode stage, and sets the valid bit to 1 to indicate fetching was done successfully.

Decode: Takes the instruction from the Fetch/Decode pipeline latch and extracts all fields based on the instruction format, either R-type or I-type: the opcode, source and destination registers, and the immediate value. If the immediate value is negative, sign-extension is performed to preserve the sign bit. It then reads the current values of the source and destination registers from the register file, and saves all extracted fields and register data into the Decode/Execute pipeline latch to prepare for the execute stage.

4. ALU Operations

The ALU was implemented as a function that accepts two 8-bit operands and an operation code, performs the requested operation, and updates a set of global status flags accordingly. The supported operations are **ADD (0)**, **SUB (1)**, **MUL (2)**, **AND (3)**, **OR (4)**, **SAL — Shift Arithmetic Left (5)**, and **SAR — Shift Arithmetic Right (6)**. (All operands and results are explicitly cast to `uint8_t` to ensure 8-bit behavior and prevent unintended sign extension or overflow from wider integer types.)

The Status Register flags — **Carry (C)**, **Overflow (V)**, **Negative (N)**, **Sign (S)**, and **Zero (Z)** — are declared as global variables and reset to zero at the beginning of every ALU call. The Carry flag is computed exclusively for addition by checking whether the unsigned sum of the two operands exceeds 8 bits. The Overflow flag is computed for both addition and subtraction: for ADD, it is set when both operands share the same sign bit but the result has a different sign bit; for SUB, it is set when the operands have different sign bits and the result carries the same sign as the subtrahend. The Negative flag is derived directly from bit 7 of the result, and the Sign flag is computed as the XOR of N and V, representing the true expected sign of the result. The Zero flag is set whenever the result equals zero.

For the remaining operations — **MUL**, **AND**, **OR**, **SAL**, and **SAR** — only the Negative and Zero flags are updated, as overflow and carry are not defined for these operations in this implementation. The arithmetic right shift (SAR) correctly uses a cast to `int8_t` before shifting to preserve the sign bit, ensuring the shift behaves as a signed operation rather than a logical one.

5. Pipeline

The pipeline implemented was a 3-stage pipeline: Instruction Fetch (IF), Instruction Decode (ID), and Execute (EX). All instructions pass through all three stages, and the pipeline can hold up to three instructions simultaneously (at the same cycle). Thus, the total number of clock cycles for a program of n instructions follows the formula $3 + (n - 1) \times 1$.

To pass information between stages, we used two **latches (pipeline registers)** :

1. IF_ID: register between fetch and decode

2. ID_EX: between decode and execute

each implemented as a struct called **Instruction** holding the instruction's opcode, register indices, immediate value, register values read from the register file, and the PC at the time of fetch. A valid flag in each latch indicates whether it holds an instruction or it's empty (bubble).

Within each cycle, the stages execute in the reverse order $EX \rightarrow ID \rightarrow IF$. This order was chosen to avoid using two copies of each latch:

1. Latch to hold the value from the previous cycle for the current stage to read,
2. Another latch to write the new value into for the next stage to prevent stage from accidentally overwriting data that another stage still needs to read in the same cycle.

By reversing the order, each stage finishes writing before the stage behind it runs, so only one latch is enough. In addition, EX runs first and writes its result into three shared variables: **fwd_result** which holds the computed value, **fwd_dest_reg** which holds the destination register index, and **fwd_valid** which signals that a fresh result is available. These values are later used in hazard detection and forwarding.

6. Memory Access and Hazards

LB and SB are the two memory access instructions in this package. LB reads a value from data memory using the immediate field as the address and loads it into the destination register, while SB takes the value stored in the source register and writes it to that address in memory. The address is treated as unsigned since the immediate is 6 bits, meaning only addresses 0 to 63 are reachable even though the data memory itself has 2048 locations. This is just a limitation of the instruction format and can't be avoided. After any instruction that has a destination register, the result gets written back to the register file at the end of EX.

For the hazards part, the main problem is that when two instructions are back to back in the pipeline, the second one reaches the decode stage while the first one is still executing, so it might read an old stale value from the register file instead of the updated one. To fix this we first detect if there's a conflict using **detectHazard()** which compares what register EX is writing to against what registers the ID instruction is trying to read. If there is a match, forwarding kicks in through **applyForwarding()** which just replaces the stale value in the latch with the correct result before execute runs, so no stall is needed. The only exception is when a LB is directly followed by an instruction that needs the loaded value, since the data from memory isn't ready in time for forwarding to work. In that case a bubble is inserted into the pipeline and everything stalls for one cycle until the value is available.